

Hands On 3

Angelo Savino

December 9, 2024

Exercise 1: Holiday Planning

The first exercise requires us to find the maximum number of attractions we can visit given a list of cities and attractions. As input, we receive the matrix M , where $M_{i,j}$ is the number of attractions that can be visited in the i -th city in the j -th day.

A requirement is that if we want to take a specific day, we have to take all the days that precede it in the row.

Given this requirement, we can encode it by performing prefix sums on each row in the matrix, in this way we obtain a new matrix M' where $M'_{i,j}$ is the number of attraction we can visit if we chose to spend j days in the i -th city.

Now we can resolve the problem by using dynamic programming. We construct a new matrix DP where $DP_{i,j}$ represent the following subproblem: "find the maximum number of attractions we can visit if we spend the first j days among the first i cities".

The matrix DP can be filled using the following recurrence:

$$DP_{i,j} \begin{cases} M_{i,j} & \text{if } i=1 \\ \max\{M_{i,j}, DP_{i-1,j}, (DP_{i-1,k} + M_{i,j-k-1}) \mid 1 \leq k < j\} & \text{otherwise} \end{cases}$$

If we have only 1 city available, then the max number of attraction we can visit is always the maximum number of attractions we can visit in that city. If we have more than one city (more precisely j cities) and only k days, we can calculate the maximum number of attractions by taking the max between the choices: not spending any day in the j -th city, spending all days in the j -th city, or spending the first k days in the previous cities and the remaining one in the j -th city.

In order to fill the matrix we need to fill each of the $n \cdot D$ cells with the solution for the corresponding subproblem, then take the cell $DP_{n,D}$ that represents the solution to the original problem.

The first row can be calculated in constant time, while all the other cells require the evaluation of the maximum over the combinations, that can be computed in $O(D)$ time, thus the total time complexity is $O(nD^2)$.

Exercise 2: Design a course

In this exercise we are given n pairs of integers (b, d) and we are required to determine the maximum number of points that can be reordered arranged such that $b_i < b_{i+1}$ and $d_i < d_{i+1}$.

Given that we are allowed to rearrange our elements, we first sort them by b to obtain an ordered array, then we perform the Longest Increasing Subsequence algorithm to determine the lis over the corresponding values of d .

This can be done by scanning the ordered array and constructing an array of dominant points, where for each element we need to perform a binary search. In this context, a point at index x dominates another point at index y if $b_x < b_y$, $d_x < d_y$ and $LIS_x \geq LIS_y$. This can be done by iterating over the positions of the original array and keeping track of the current dominant points. Possible updates of the points can be performed by employing binary search. After iterating over all the array, the length of our array of dominating points is also our answer for the problem.

The array of points can be sorted in $O(n \log(n))$, then the LIS algorithm requires at most one binary search for each element of the sorted array, thus the total time complexity is $O(n \log(n))$.